מדריך לימוד מרוכז ומעמיק

רשתות נוירונים ואימון מודלים

Weights, Bias, Activation Functions, Gradient, Loss, Backpropagation, Classification, Overfitting, SGD, BGD ו-Mini-Batch - מההיגיון הבסיסי ועד תהליך האימון המלא.

מטרת המסמך: להבין את החומר ברמה שמאפשרת להסביר אותו בעל-פה, לא רק לשנן הגדרות. בכל פרק יש הסבר אינטואיטיבי, נוסחה בסיסית, דוגמה ותשובה שמתאימה לשאלה מקצועית.

מפת הדרך - לפני שמתחילים

כמעט כל החומר ברשימה מתחבר לסיפור אחד פשוט: **רשת נוירונים מקבלת קלט, מחשבת תחזית, בודקת כמה טעתה, ומתקנת את הפרמטרים שלה כך שבפעם הבאה תטעה פחות.**



המשפט שצריך לזכור: האימון הוא תהליך של התאמת משקלים ו-Biases כדי למזער את פונקציית ה-Loss.

מהו נוירון מלאכותי?

נוירון מלאכותי מקבל מספר קלטים. לכל קלט יש משקל, לאחר מכן מחברים Bias ומעבירים את התוצאה דרך פונקציית אקטיבציה.

$$z = x_1w_1 + x_2w_2 + \ldots + x_nw_n + b$$
$$\text{output} = f(z)$$

כאן x הם הקלטים, w הם המשקלים, b הוא ה-Bias, ו-f היא פונקציית האקטיבציה.

תוכן העניינים

1. משקלים - Weights
2. Bias
3. Activation Function
4. Step / ReLU / Sigmoid / Softmax
5. Threshold
6. Gradient
7. Loss Function
8. Backpropagation
9. Classification
10. רשת כפונקציה מרובת משתנים
11. פונקציות אקטיבציה בשכבות
12. שכבת הקלט
13. Overfitting
14. תהליך אימון מלא
15. BGD / SGD / Mini-Batch
16. Epoch / Learning Rate / Optimizer
17. שאלות ותשובות לחזרה

1. משקלים - Weights

המשקל הוא מספר שקובע **כמה חזק קלט מסוים משפיע על תוצאת הנוירון**. אם לקלט מסוים יש משקל גדול בערך מוחלט, שינוי קטן בקלט הזה ישפיע יותר על הנוירון. משקל חיובי דוחף את התוצאה לכיוון אחד; משקל שלילי דוחף בכיוון ההפוך.

דוגמה: נניח שאנחנו מנסים לזהות אם אירוע אבטחה חשוד. הקלטים הם מספר ניסיונות התחברות כושלים וחריגה בשעת הכניסה. אם המודל למד שחריגה בשעה היא אינדיקציה חזקה יותר, הוא עשוי לתת לה Weight גדול יותר.

$$z = (x_1 \cdot w_1) + (x_2 \cdot w_2) + b$$

אם $w_1 = 0.2$ ו-$w_2 = 2.3$, הקלט השני משמעותי יותר בחישוב.

מה המודל בעצם "לומד"?

כאשר אומרים שמודל "לומד", הכוונה העיקרית היא שהוא משנה את ערכי ה-Weights וה-Biases. בתחילת האימון הם מאותחלים לערכים התחלתיים; לאחר כל סבב של חיזוי וחישוב טעות, הם משתנים מעט. אחרי הרבה מאוד עדכונים נוצרת קונפיגורציה של פרמטרים שמבצעת את המשימה היטב.

תשובה קצרה: Weight הוא פרמטר נלמד שקובע את עוצמת ההשפעה של קלט או חיבור על הנוירון.

2. Bias

Bias הוא מספר שמתווסף לסכום המשוקלל של הקלטים לפני האקטיבציה. תפקידו לתת לנוירון גמישות להזיז את נקודת ההחלטה גם כאשר כל הקלטים אפס.

$$z = \Sigma(x_i w_i) + b$$

דמיין קו ישר. המשקל משפיע על השיפוע; ה-Bias מזיז את הקו למעלה או למטה. ברשתות עמוקות זה מאפשר לכל נוירון למקם את הגבול שלו במקום שימושי יותר.

דיוק חשוב: Bias אינו "ה-Threshold" עצמו, אבל הוא קשור ישירות למיקום גבול ההחלטה. אפשר לחשוב עליו כפרמטר שמזיז את נקודת הסף של הנוירון.

3. פונקציית אקטיבציה - Activation Function

לאחר החישוב הליניארי של הנוירון מתקבל מספר z. פונקציית האקטיבציה מקבלת אותו ומחליטה מה יהיה הפלט של הנוירון.

$$output = f(z)$$

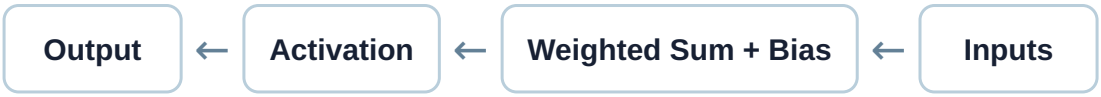
למה בכלל צריך פונקציית אקטיבציה?

אם כל שכבה ברשת הייתה עושה רק כפל וחיבור, כל הרשת - גם אם היו בה מאות שכבות - הייתה ניתנת לצמצום לפונקציה ליניארית אחת. המשמעות היא שהיא לא הייתה מסוגלת ללמוד גבולות מורכבים, צורות, דפוסים לא-ליניאריים וקשרים מורכבים בין משתנים.

הנקודה המרכזית: פונקציית האקטיבציה מכניסה Non-Linearity. בלי אי-ליניאריות אין לרשת עמוקה כוח ייצוג משמעותי מעבר למודל ליניארי.

מה קורה בפועל?

לכל נוירון יש ערך לפני אקטיבציה. פונקציית האקטיבציה ממפה אותו לערך חדש. הערך הזה נשלח לנוירונים בשכבה הבאה. כך כל שכבה מבצעת טרנספורמציה על הייצוג שקיבלה.



מתי בוחרים פונקציה מסוימת?

הבחירה תלויה בתפקיד השכבה. שכבות נסתרות צריכות פונקציות שמתאימות ללמידה יציבה ויעילה; שכבת הפלט צריכה פונקציה שמתאימה לסוג הבעיה - למשל Sigmoid ל-Binary Classification או Softmax ל-Multiclass.

4. סוגי פונקציות אקטיבציה

Step Function

פונקציה בינארית פשוטה: אם הקלט עבר סף - מחזירים 1, אחרת 0.

$$f(x) = 1 \text{ אם } x > \text{threshold}, 0 \text{ אחרת}$$

היא אינטואיטיבית, אבל לא מתאימה לרוב האימון המודרני משום שהנגזרת שלה לא שימושית כמעט בכל התחום. לכן קשה לבצע עליה Gradient Descent.

ReLU

$$\text{ReLU}(x) = \max(0, x)$$

אם הקלט שלילי - הפלט 0. אם חיובי - משאירים אותו כפי שהוא. היא פשוטה, מהירה וחישובית, ובמשך שנים הייתה ברירת מחדל נפוצה בשכבות נסתרות.

יתרון: מאפשרת Gradient נוח יחסית עבור ערכים חיוביים. **חיסרון:** נוירון יכול "למות" אם הוא נשאר באזור השלילי ומפסיק לקבל Gradient משמעותי.

Sigmoid

$$\sigma(x) = 1 / (1 + e^{-x})$$

ממפה כל מספר לטווח שבין 0 ל-1. לכן מתאימה במיוחד לשכבת פלט של Binary Classification.

חיסרון מרכזי: בקצוות הפונקציה נעשית שטוחה, ולכן ה-Gradient נעשה קטן מאוד - תופעה שיכולה לתרום ל-Vanishing Gradients.

Softmax

מקבלת וקטור של ציונים וממירה אותו להתפלגות שערכיה חיוביים וסכומם 1. לדוגמה:

מחלקה	ציון לפני Softmax	הסתברות אחרי Softmax
חתול	4.2	0.86
כלב	2.1	0.11
סוס	0.4	0.03

5. Threshold - סף החלטה

Threshold הוא ערך סף שלפיו הופכים ציון רציף להחלטה דיסקרטית. למשל, אם המודל מחזיר הסתברות של 0.82 ל-Malicious והסף הוא 0.5, נסווג את האירוע כזדוני.

דוגמה אבטחתית: אפשר להעלות Threshold ל-0.9 כדי לצמצם False Positives, במחיר של החמצת חלק מהאיומים. לכן Threshold הוא גם כלי עסקי/תפעולי ולא רק מתמטי.

חשוב: Threshold אינו זהה לפונקציית האקטיבציה. אפשר להשתמש ב-Sigmoid כדי לקבל הסתברות, ואז להחיל Threshold בנפרד לצורך החלטה.

6. Gradient - גרדיאנט

Gradient עונה על השאלה: **אם אשנה מעט פרמטר מסוים, לאיזה כיוון ובאיזו עוצמה ה-Loss ישתנה?**

אם יש לנו פונקציית Loss שתלויה בהרבה פרמטרים, הגרדיאנט הוא וקטור של נגזרות חלקיות:

$$\nabla L = [\partial L/\partial w_1, \partial L/\partial w_2, \ldots, \partial L/\partial w_n]$$

כל רכיב מספר לנו מה השפעת שינוי קטן ב-Weight מסוים על ה-Loss.

איך משתמשים בגרדיאנט?

הגרדיאנט מצביע לכיוון שבו הפונקציה עולה הכי מהר. אם רוצים להקטין Loss, הולכים בכיוון ההפוך.

$$w_{new} = w_{old} - learning_rate \times gradient$$

אינטואיציה: תחשוב שאתה על הר ורוצה להגיע לעמק. הגרדיאנט אומר לך איפה העלייה התלולה; אתה הולך לכיוון ההפוך כדי לרדת.

7. Loss Function - פונקציית הפסד

ה-Loss הוא מספר שמודד **כמה התחזית של המודל גרועה ביחס לתשובה הרצויה**. האימון כולו מכוון להקטין את הערך הזה.

דוגמה: האמת היא "חתול", אבל המודל נותן 20% חתול ו-80% כלב. ה-Loss יהיה גבוה. אם המודל נותן 99% חתול, ה-Loss יהיה נמוך.

למה לא מספיק Accuracy?

Accuracy הוא מדד בדיקה טוב, אבל הוא לא תמיד רציף ולא תמיד מספק Gradient שימושי. Loss נבנה כך שיהיה ניתן לגזור אותו ולהשתמש בו לאימון.

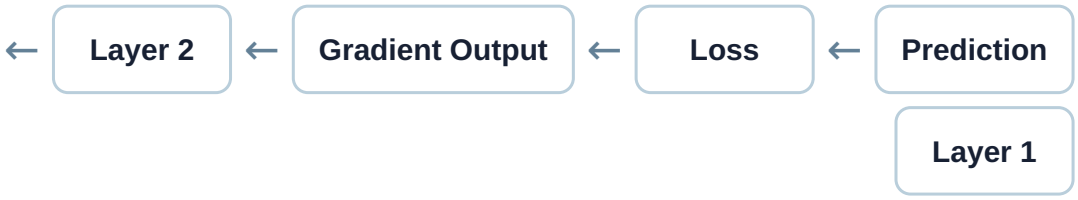
דוגמאות לפונקציות Loss

- **Mean Squared Error:** נפוצה ב-Regression; מודדת ריבוע ההפרש בין תחזית לערך אמיתי.
- **Binary Cross-Entropy:** נפוצה ב-Binary Classification.
- **Categorical Cross-Entropy:** נפוצה ב-Multiclass Classification יחד עם Softmax.

תשובה קצרה: Loss Function היא הפונקציה שמתרגמת את איכות התחזית למספר שאפשר למזער באמצעות Gradient Descent.

8. Backpropagation - מעבר לאחור

Backpropagation הוא האלגוריתם שמחשב את ה-Gradients של ה-Loss ביחס לכל הפרמטרים ברשת. הוא עובר מהפלט לאחור דרך השכבות ומשתמש בכלל השרשרת - Chain Rule.



דיוק חשוב: Backpropagation מחשב את ה-Gradients. ה-Optimizer הוא זה שמשתמש בהם כדי לעדכן בפועל את המשקלים.

איך Backpropagation עובד - קצת יותר עמוק

נניח שיש לנו שרשרת של פונקציות: הקלט נכנס לשכבה ראשונה, משם לשנייה, ומשם לפלט. ה-Loss תלוי בסוף בכל משקל שנמצא בדרך.

$$L = L(f_3(f_2(f_1(x))))$$

כדי לדעת כיצד Weight מוקדם ברשת משפיע על ה-Loss, משתמשים בכלל השרשרת:

$$\partial L/\partial w = (\partial L/\partial output) \times (\partial output/\partial hidden) \times (\partial hidden/\partial w)$$

כל שכבה מקבלת Gradient מהשכבה שאחריה, מחשבת את התרומה שלה, ושולחת Gradient אחורה. כך אפשר לחשב בצורה יעילה Gradients עבור מיליוני או מיליארדי פרמטרים.

למה זה כל כך חשוב?

בלי Backprop היינו צריכים לשנות כל Weight בנפרד, להריץ את כל המודל ולבדוק אם השתפר - תהליך בלתי אפשרי עבור רשתות גדולות. Backprop מנצל את מבנה הרשת ומחשב את כל הנגזרות בצורה יעילה.

הקשר ל-GPU: Forward ו-Backward כוללים כמויות אדירות של כפל מטריצות ופעולות וקטוריות. לכן הם מתאימים מאוד לביצוע מקבילי על GPU.

Forward לעומת Backward

שלב	מה קורה?
Forward Pass	הקלט עובר קדימה ומייצר Prediction.
Loss	נמדדת הטעות בין Prediction ל-Target.
Backward Pass	מחושבים Gradients לכל הפרמטרים.
Optimizer Step	המשקלים מתעדכנים לפי ה-Gradients.

9. Classification - בעיית סיווג

Classification היא בעיה שבה המודל צריך להחליט לאיזו קטגוריה קלט שייך.

Multiclass Classification

אחת מתוך כמה מחלקות.

דוגמאות: חתול / כלב / סוס / אריה.

פלט נפוץ: Softmax.

Binary Classification

שתי מחלקות בלבד.

דוגמאות: זדוני / תקין, ספאם / לא ספאם, חתול / לא חתול.

פלט נפוץ: Sigmoid.

ומה לגבי Multi-label?

לפעמים קלט יכול להשתייך לכמה קטגוריות במקביל - למשל תמונה יכולה להכיל גם אדם וגם רכב וגם כלב. במצב כזה נהוג לעיתים להשתמש ב-Sigmoid נפרד לכל מחלקה, ולא ב-Softmax שמכריח סכום הסתברויות של 1.

10. הקשר בין רשת נוירונים לפונקציה מרובת משתנים

רשת נוירונים היא, מבחינה מתמטית, פונקציה גדולה שמקבלת הרבה משתנים ומחזירה פלט.

$$y = f(x_1, x_2, \ldots, x_n ; \theta)$$

θ מייצג את כל הפרמטרים הנלמדים - Weights ו-Biases. אם יש מיליוני או מיליארדי פרמטרים, אנחנו למעשה עובדים עם פונקציה במרחב רב-ממדי מאוד.

אז מהו האימון מבחינה מתמטית?

אנחנו מחפשים את ערכי θ שימזערו את ה-Loss:

$$\theta^* = \text{argmin } L(\theta)$$

כל Weight הוא "ציר" אחד במרחב הפרמטרים. ה-Gradient אומר לנו באיזה כיוון במרחב הזה ה-Loss גדל, וה-Optimizer צועד לכיוון ההפוך.

11. האם בכל הנוירונים באותה שכבה משתמשים באותה פונקציית אקטיבציה?

בדרך כלל כן. ארכיטקטורות סטנדרטיות מגדירות פונקציית אקטיבציה ברמת השכבה, ולכן כל הנוירונים באותה שכבה משתמשים באותה פונקציה.

> לדוגמה: Dense Layer עם 256 נוירונים ו-ReLU - כל 256 הנוירונים מפעילים ReLU על הפלט שלהם.

אפשר טכנית לבנות שכבה שבה נוירונים שונים משתמשים בפונקציות שונות, אבל זה אינו הדפוס המקובל ברשתות רגילות.

12. האם בין שכבות שונות משתמשים בפונקציות שונות?

בהחלט. זה אפילו נפוץ מאוד, כי לשכבות שונות יש תפקידים שונים.



למשל, ברשת Classification רב-מחלקתית, השכבות הפנימיות יכולות להשתמש ב-ReLU או GELU, ושכבת הפלט ב-Softmax.

13. האם לשכבת הקלט יש Activation Function?

בדרך כלל לא. שכבת הקלט אינה שכבת חישוב במובן הקלאסי; היא מייצגת את הנתונים שנכנסים למודל. החישוב הראשון מתחיל בשכבה שאחריה.

> **הבחנה:** אפשר לעשות Preprocessing, Normalization או Embedding לקלט, אבל זה לא אותו דבר כמו לומר שלנוירוני הקלט עצמם יש Activation Function.

14. Overfitting - התאמת יתר

Overfitting קורה כאשר המודל לומד את דוגמאות האימון בצורה טובה מדי - כולל רעש, פרטים מקריים ודפוסים שלא באמת מייצגים את העולם - ולכן הוא מתקשה להכליל על נתונים חדשים.

דוגמה: Training Accuracy = 99.8%, אבל Validation Accuracy = 73%. המודל מצוין על מה שכבר ראה, אבל חלש על מידע חדש.

איך מזהים?

- Training Loss ממשיך לרדת.
- Validation Loss מפסיק לרדת ואף מתחיל לעלות.
- פער הולך וגדל בין ביצועים על Train לבין Validation/Test.

למה זה קורה?

- מודל מורכב מדי ביחס לכמות הדאטה.
- מעט מדי דוגמאות או דאטה לא מגוון.
- יותר מדי Epochs.
- רעש או Labels לא מדויקים.

איך מתמודדים?

שיטה	רעיון
יותר Data	מכריח את המודל ללמוד דפוסים כלליים יותר.
Data Augmentation	יוצרים וריאציות של דוגמאות קיימות.
Dropout	מכבים אקראית חלק מהנוירונים בזמן אימון.
Regularization	מענישים פרמטרים גדולים מדי, למשל L1/L2.
Early Stopping	עוצרים כשביצועי ה-Validation מפסיקים להשתפר.
מודל קטן יותר	מפחיתים יכולת שינון מיותרת.

תשובה קצרה: Overfitting הוא מצב שבו המודל למד את ה-Training Set יותר מדי טוב ופוגע ביכולת ההכללה שלו לנתונים שלא ראה.

15. תהליך האימון - מההתחלה עד הסוף

שלב 1 - הכנת Data

אוספים דוגמאות ומחלקים לרוב ל-Training, Validation ו-Test. ה-Training משמש לעדכון הפרמטרים; Validation לבחירת Hyperparameters ומעקב אחר Overfitting; Test להערכה סופית.

שלב 2 - Initialization

מאתחלים Weights לערכים התחלתיים מתאימים. לא מאתחלים את כולם לאותו ערך, אחרת נוירונים עלולים ללמוד בדיוק אותו דבר.

שלב 3 - בחירת Mini-Batch

לוקחים קבוצה קטנה מתוך הדאטה, למשל 32, 64 או 128 דוגמאות.

שלב 4 - Forward Pass

ה-Mini-Batch עובר דרך כל השכבות. בכל שכבה מתבצעים כפל מטריצות, הוספת Bias ואקטיבציה. מתקבלת תחזית.

שלב 5 - Loss

משווים את התחזית ל-Target ומחשבים Loss ממוצע עבור ה-Batch.

שלב 6 - Backpropagation

מחשבים Gradients של ה-Loss ביחס לכל Weight ו-Bias.

שלב 7 - Optimizer Step

ה-Optimizer מעדכן את הפרמטרים, למשל באמצעות SGD או Adam.

שלב 8 - Repeat

עוברים ל-Mini-Batch הבא. כאשר עברנו על כל ה-Training Set השלמנו Epoch אחד. ממשיכים כמה Epochs עד שהמודל מתכנס או נעצר לפי קריטריון.

הסיפור במשפט: Forward נותן תחזית, Loss מודד טעות, Backprop מחשב מי אחראי לטעות, וה-Optimizer משנה את הפרמטרים.

16. BGD, SGD ו-Mini-Batch

כל שלוש השיטות עונות על אותה שאלה: **על כמה דוגמאות מחשבים Gradient לפני שמעדכנים את המשקלים?**

שיטה	כמה דוגמאות לעדכון	יתרון	חיסרון
BGD - Batch Gradient Descent	כל ה-Training Set	Gradient יציב ומדויק יחסית	כבד מאוד בזיכרון ובזמן
SGD קלאסי	דוגמה אחת	עדכונים תכופים, מעט זיכרון	Gradient רועש ולא יציב
Mini-Batch	קבוצה קטנה, למשל 32-512	איזון מצוין + ניצול GPU	דורש בחירת Batch Size

למה Mini-Batch הוא ברירת המחדל המעשית?

הוא מאפשר לחשב הרבה דוגמאות במקביל על GPU, לשמור על Gradient יציב יחסית, ולעדכן את המודל בתדירות סבירה. לכן רוב ספריות ה-Deep Learning משתמשות ב-Mini-Batch Training.

SGD - הערת דיוק

בשפה מקצועית אנשים אומרים לעיתים "SGD" גם כאשר משתמשים למעשה ב-Mini-Batches. SGD הוא שם משפחה של שיטות שבהן משתמשים בדגימה סטוכסטית של הדאטה ולא בכל ה-Dataset בכל צעד.

Learning Rate

Learning Rate קובע את גודל הצעד שבו מעדכנים את הפרמטרים.

$$w_{new} = w_{old} - \eta \cdot \partial L/\partial w$$

אם הוא גדול מדי - אפשר לדלג מעל המינימום ולהיות לא יציבים. אם קטן מדי - הלמידה איטית מאוד ועלולה להיתקע.

17. Optimizers - מי באמת משנה את המשקלים?

SGD

הגרסה הבסיסית מעדכנת כל Weight בכיוון ההפוך ל-Gradient. אפשר להוסיף Momentum כדי לשמר "תנופה" ולצמצם תנודות.

Adam

Optimizer נפוץ מאוד שמשלב מידע על ממוצעי Gradients וממוצעי ריבועי Gradients כדי לתת Learning Rate אדפטיבי לכל פרמטר.

דיוק חשוב: Backpropagation אינו Optimizer. Backprop מחשב Gradients; SGD/Adam משתמשים בהם כדי לבצע Update.

Batch-ו Epoch, Iteration

- **Batch:** קבוצת דוגמאות שנכנסות יחד למודל.
- **Iteration / Step:** Forward + Backward + Update אחד.
- **Epoch:** מעבר מלא אחד על כל ה-Training Set.

דוגמה: יש 10,000 דוגמאות ו-Batch Size של 100. בכל Epoch יהיו 100 Steps. אם מאמנים 20 Epochs, יהיו בערך 2,000 עדכוני פרמטרים.

Training לעומת Inference

Training	Inference
Forward + Loss + Backward + Update	רק Forward Pass
משנה Weights	Weights נשארים קבועים
דורש יותר זיכרון וחישוב	בדרך כלל קל וזול יותר
צריך לשמור Activations ו-Gradients	אין צורך ב-Gradients

18. דוגמה מלאה - נוירון שמסווג אירוע אבטחה

נניח שיש לנו שני Features:

- x_1 = מספר ניסיונות התחברות כושלים.
- x_2 = אינדיקציה להתחברות בשעה חריגה.

המשקלים כרגע:

$$w_1 = 0.4,\ w_2 = 1.2,\ b = -2.0$$

עבור אירוע עם $x_1 = 5$ ו-$x_2 = 1$:

$$z = 5 \times 0.4 + 1 \times 1.2 - 2.0 = 1.2$$

נעביר דרך Sigmoid:

$$\sigma(1.2) \approx 0.77$$

אם Threshold = 0.5, המודל מסווג את האירוע כחשוד.

עכשיו נניח שהתשובה האמיתית היא "תקין"

המוצר טעה. ה-Loss יהיה יחסית גבוה. Backprop יחשב איך כל Weight תרם לטעות. ייתכן שיתברר שהמשקל של "שעה חריגה" גבוה מדי. ה-Optimizer יקטין אותו מעט.

$$w_2: 1.20 \rightarrow 1.17$$

אחרי אלפי דוגמאות כאלה, המשקלים יתכנסו לערכים שנותנים Classification טוב יותר.

> **זו רשת לומדת:** לא כותבים לה ידנית מהו המשקל הנכון. מגדירים Objective, נותנים דוגמאות, והיא מתאימה את הפרמטרים על בסיס Gradient.

19. שאלות קצרות - תשובות שמומלץ לדעת בעל-פה

מה זה Weight?

פרמטר נלמד שקובע כמה קלט או חיבור משפיע על תוצאת הנוירון.

מה זה Bias?

פרמטר נוסף שמוזיז את פונקציית הנוירון ואת נקודת ההחלטה, מעבר להשפעת הקלטים עצמם.

למה צריך Activation?

כדי להכניס אי-ליניאריות; בלי Activation לא-ליניארי, רשת עמוקה הייתה שקולה למודל ליניארי.

מה זה Gradient?

וקטור נגזרות שמראה כיצד שינוי קטן בכל פרמטר ישפיע על ה-Loss.

מה זה Loss?

פונקציה שמודדת כמה התחזית של המודל רחוקה מהמטרה ואותה מנסים למזער באימון.

מה עושה Backprop?

מחשב את ה-Gradients של ה-Loss ביחס לפרמטרים באמצעות מעבר לאחור ו-Chain Rule.

מי משנה את ה-Weights בפועל?

Optimizer כמו SGD או Adam.

מה ההבדל בין SGD ל-BGD?

BGD מחשב Gradient על כל הדאטה לפני Update; SGD קלאסי עושה Update לכל דוגמה; Mini-Batch עושה Update לקבוצה קטנה.

מה זה Overfitting?

כאשר המודל מצליח מאוד על Training Data אבל לא מכליל לנתונים חדשים.

למה GPU מתאים לאימון?

כי Forward ו-Backward כוללים המון פעולות מטריצה שניתנות למקביליות גבוהה.

20. טעויות נפוצות שכדאי להימנע מהן

- **"Bias הוא Threshold"** - לא בדיוק. Bias משפיע על מיקום גבול ההחלטה; Threshold הוא כלל החלטה שמוחל על פלט.
- **"Backprop מעדכן את המשקלים"** - Backprop מחשב Gradients. Optimizer מעדכן.
- **"Softmax תמיד נדרש לסיווג"** - לא. הוא מתאים בעיקר ל-Multiclass שבו רוצים מחלקה אחת מתוך כמה.
- **"SGD אומר תמיד Sample אחד"** - בהגדרה הקלאסית כן, אבל בשיח מעשי לעיתים קוראים SGD גם ל-Mini-Batch SGD.
- **"אם Training Accuracy גבוה, המודל טוב"** - לא בהכרח. צריך לבדוק Validation/Test כדי לוודא Generalization.
- **"יותר Epochs תמיד טוב"** - לא. יותר מדי עלול להוביל ל-Overfitting.
- **"Activation Function קובעת אם הנוירון ON/OFF"** - זו אינטואיציה היסטורית, אבל ברשתות מודרניות הפלט לרוב רציף ולא בינארי.

21. סיכום תהליך האימון במשפטים

1. מאתחלים פרמטרים.
2. לוקחים Mini-Batch.
3. מבצעים Forward Pass.
4. מחשבים Loss.
5. מבצעים Backpropagation.
6. מקבלים Gradients.
7. Optimizer מעדכן Weights ו-Biases.
8. חוזרים על התהליך שוב ושוב.
9. בודקים Validation כדי למנוע Overfitting.
10. בסיום משתמשים במודל ל-Inference ללא Backprop.

אם אתה מבין את השרשרת הזו ואת התפקיד של כל מושג בתוכה - אתה מבין את הבסיס של אימון רשתות נוירונים.

22. דף חזרה מהיר לפני שיחה או מבחן

מושג	משפט אחד לזכור
Weight	כמה חזק הקלט משפיע.
Bias	מזיז את נקודת ההחלטה.
Activation	מכניסה אי-ליניאריות.
ReLU	שלילי הופך ל-0, חיובי נשאר.
Sigmoid	ממפה ל-0 עד 1; שימושית לבינארי.
Softmax	הופכת ציונים להתפלגות על כמה מחלקות.
Threshold	סף שממנו הופכים ציון להחלטה.
Loss	כמה המודל טעה.
Gradient	איך שינוי פרמטר ישפיע על ה-Loss.
Backprop	מחשב Gradients לאחור.
Optimizer	מעדכן את הפרמטרים.
Mini-Batch	קבוצת דוגמאות לעדכון אחד.
Epoch	מעבר מלא אחד על כל Training Set.
Overfitting	מצוין על Train, חלש על מידע חדש.

תשובת-על: רשת נוירונים היא פונקציה מרובת משתנים ופרמטרים. בזמן אימון היא מנסה למזער Loss. Backprop מחשב Gradients, ו-Optimizer משתמש בהם כדי לעדכן Weights ו-Biases, בדרך כלל על Mini-Batches, עד שהמודל מכליל היטב לנתונים חדשים.

המסמך נכתב כחומר לימוד מעשי: הוא מפשט חלק מהפרטים המתמטיים כדי לבנות הבנה אינטואיטיבית נכונה, ובו בזמן שומר על ההבחנות המקצועיות החשובות.